

CMPE679: Deep Learning  
Homework 1  
MNIST Classification Using PyTorch

Gloria Mbaka  
Performed: 1/28/2026  
Submitted: 1/29/2026

Instructor: Dr. Liu, Dongfang  
TA: Huang, Renzhong; and Renjen, Ishan

By submitting this report, I attest that I neither have given nor have received any unauthorized assistance (including writing, collecting data, plotting figures, tables, or graphs, or using previous student reports as a reference), and I further acknowledge that giving or receiving such assistance will result in a failing grade for this course.

Your Signature: CGM

# 1. Introduction

This homework focused on the implementation and analysis of a neural network for handwritten digit classification using the MNIST dataset and the PyTorch deep learning framework. The primary objective of the assignment was to understand the fundamental components of a supervised learning pipeline, including data preprocessing, model design, training using gradient-based optimization, and performance evaluation. The MNIST dataset consists of grayscale images of handwritten digits from 0 to 9, with each image represented as a  $28 \times 28$  matrix of pixel intensity values. These images were flattened into 784-dimensional input vectors and used to train a fully connected neural network. The model architecture consisted of an input layer, a single hidden layer with ReLU activation, and an output layer producing class scores for the ten digit categories. The implementation was carried out through a sequence of guided steps labeled TODO 0 through TODO 6, which included dataset inspection, model definition, loss and optimizer selection, training loop construction, evaluation, and result visualization. Training was performed over multiple epochs using mini-batch gradient descent with the Adam optimizer and cross-entropy loss. Model performance was evaluated using classification accuracy on a held-out test set, and training behavior was analyzed using loss and accuracy plots across epochs. This report documents the implementation process, explains the reasoning behind key design choices, and analyzes the observed training and evaluation results.

## 2. Dataset and Problem Definition

The MNIST dataset was used as the benchmark dataset for this assignment. It consists of a large collection of grayscale images representing handwritten digits from 0 through 9. Each image has a fixed resolution of  $28 \times 28$  pixels, where each pixel value corresponds to the intensity of the grayscale color (Figure 1). The dataset is divided into two subsets: a training set used for learning model parameters and a test set used for evaluating generalization performance. For this classification task, each input image was treated as a single data sample. Since fully connected neural networks operate on one-dimensional vectors, each  $28 \times 28$  image was flattened into a 784-dimensional input vector before being passed to the model. The corresponding output label for each input is an integer between 0 and 9, representing the digit class associated with the image.

The problem was formulated as a supervised multi-class classification task. Given an input vector of pixel values, the objective of the model is to predict the correct digit class by assigning the highest score to the corresponding output category. The model outputs a vector of ten values, each representing the unnormalized score (logit) for one digit class. The predicted class is determined by selecting the index of the maximum output value. To facilitate efficient training, the dataset was processed in mini-batches of 64 pixels using PyTorch's data loading utilities. Mini-batch processing allows the model to update its parameters incrementally and improves computational efficiency compared to processing the entire dataset at once. The use of a separate test dataset ensures that model performance is evaluated on unseen data, providing a measure of the model's ability to generalize beyond the training samples.

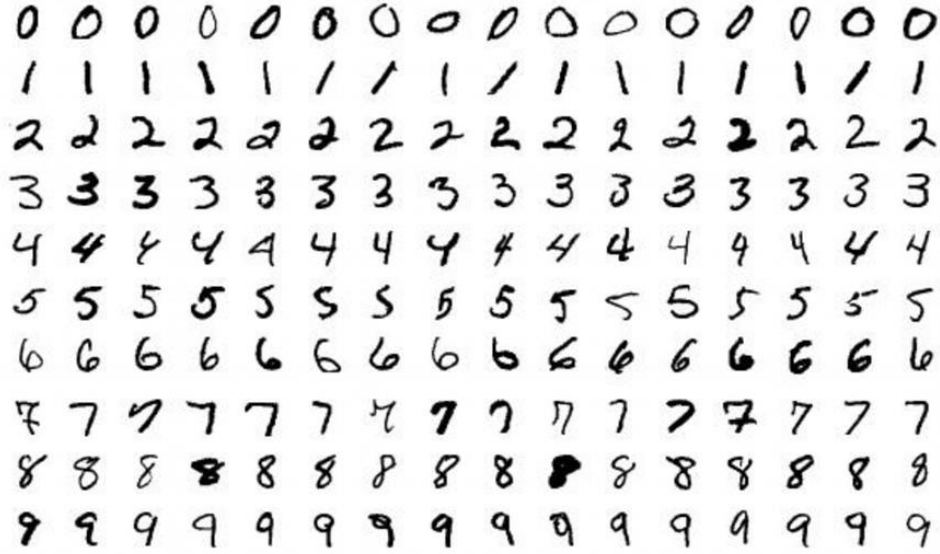


Figure 1: MNIST Dataset

### 3. Model Architecture Design

The neural network is a fully connected feedforward network composed of one hidden layer. The purpose of this architecture is to model the relationship between the input pixel values of an MNIST image and its corresponding digit label through a sequence of linear transformations and nonlinear activations.

Each MNIST image is represented as a grayscale matrix of size  $28 \times 28$ . Prior to being passed into the network, the image is flattened into a vector

$$\mathbf{x} \in \mathbb{R}^{784}$$

where each element corresponds to the intensity of a single pixel.

#### 3.1 First (Hidden) Layer

The first layer performs an affine transformation of the input vector followed by a nonlinear activation. This operation is expressed as:

$$\mathbf{z}_1 = \mathbf{W}_1 \mathbf{x} + \mathbf{b}_1$$

where

$$\mathbf{W}_1 \in \mathbb{R}^{128 \times 784}, \mathbf{b}_1 \in \mathbb{R}^{128}$$

The output of this linear transformation is then passed through the Rectified Linear Unit (ReLU) activation function:

$$\mathbf{a}_1 = \text{ReLU}(\mathbf{z}_1)$$

with ReLU defined element-wise as:

$$\text{ReLU}(z) = \max(0, z)$$

The ReLU activation introduces nonlinearity into the model, allowing it to learn complex decision boundaries that cannot be represented by a purely linear classifier.

### 3.2 Output Layer

The activated hidden-layer output is then passed to the final fully connected layer:

$$\mathbf{z}_2 = \mathbf{W}_2 \mathbf{a}_1 + \mathbf{b}_2$$

where

$$\mathbf{W}_2 \in \mathbb{R}^{10 \times 128}, \mathbf{b}_2 \in \mathbb{R}^{10}$$

The resulting vector

$$\mathbf{z}_2 \in \mathbb{R}^{10}$$

contains the **logits**, or unnormalized class scores, corresponding to the ten digit classes (0–9). No activation function is applied at this stage, as the logits are passed directly to the loss function during training.

### 3.3 Overall Model Mapping

Combining the operations of both layers, the network computes the following function:

$$f(\mathbf{x}) = \mathbf{W}_2 \text{ReLU}(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$$

This function maps a 784-dimensional input vector to a 10-dimensional output vector representing class scores. A schematic representation of this architecture is shown in **Figure 2**, illustrating the dimensionality changes from input to output through the network.

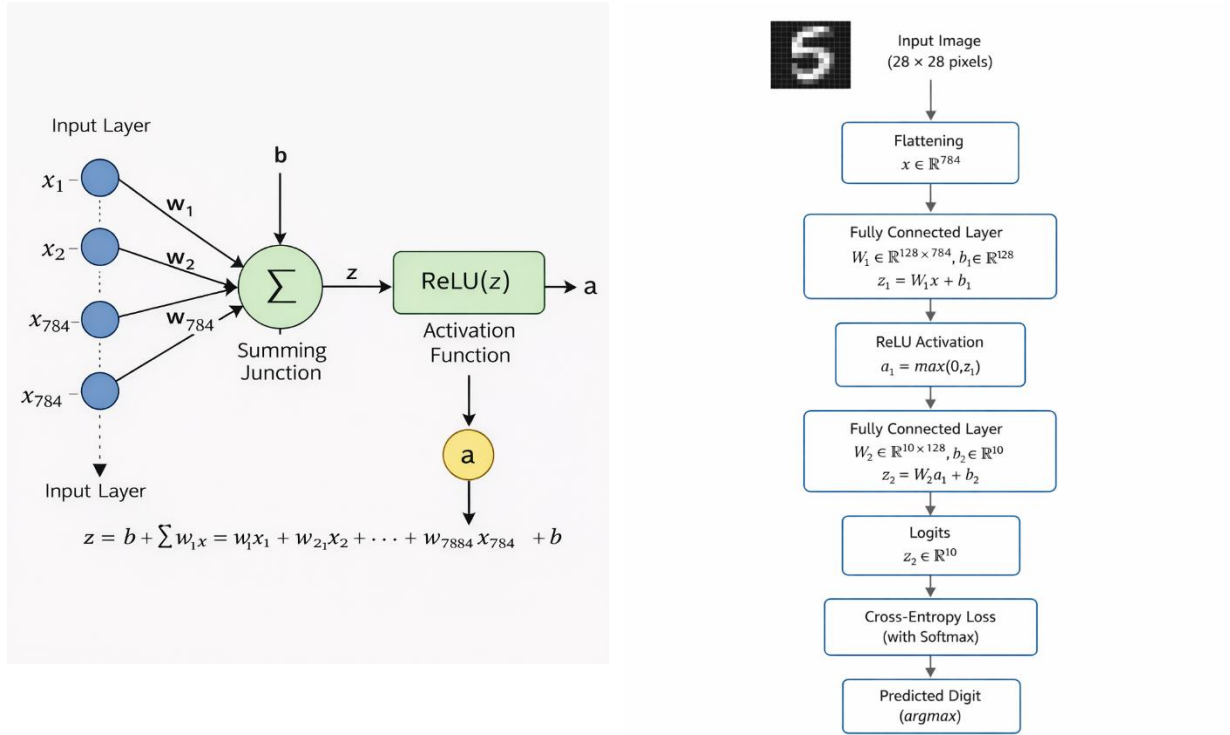


Figure 2: Perceptron model of a single neuron used in the fully connected neural network. Input features  $x_i$  are weighted, summed with a bias term  $b$  to form  $z$ , and passed through a ReLU activation function to produce the neuron output  $a$ .

## 4. Training Methodology

The neural network was trained using supervised learning, where each input image is paired with a known digit label. Training consists of iteratively adjusting the network's weights and biases in order to minimize a loss function that quantifies the difference between the model's predictions and the true labels. This optimization is performed using gradient-based learning through backpropagation.

### 4.1 Loss Function

The cross-entropy loss function was used to train the model. For a single input sample, the network produces a vector of logits

$$\mathbf{z} \in \mathbb{R}^{10}$$

where each element represents the unnormalized score for one digit class. These logits are internally converted into class probabilities using the softmax function:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{10} e^{z_j}}$$

Given the true class label  $y$ , the cross-entropy loss for one sample is defined as:

$$\mathcal{L} = -\log(p_y)$$

This loss penalizes the model when it assigns a low probability to the correct class. During training, the loss is computed for each sample in a mini-batch and averaged to produce the batch loss value. In this implementation, a mini-batch size of 64 samples was used.

In PyTorch, this operation is implemented using:

```
criterion = nn.CrossEntropyLoss()
loss = criterion(outputs, labels)
```

## 4.2 Gradient Computation and Backpropagation

To minimize the loss, gradients of the loss with respect to each trainable parameter are computed using backpropagation. Consider a neuron with input  $x_i$ , weight  $w_i$ , bias  $b$ , and pre-activation value:

$$z = \sum_i w_i x_i + b$$

The gradient of the loss with respect to a weight  $w_i$  is obtained using the chain rule:

$$\frac{\partial \mathcal{L}}{\partial w_i} = \frac{\partial \mathcal{L}}{\partial z} \cdot x_i$$

Similarly, the gradient with respect to the bias is:

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial z}$$

For the combination of softmax and cross-entropy loss, the gradient of the loss with respect to the logits simplifies to:

$$\frac{\partial \mathcal{L}}{\partial z_k} = p_k - y_k$$

where  $y_k$  is the one-hot encoded true label. This simplification improves numerical stability and computational efficiency.

In the implementation, gradient computation is triggered automatically by:

```
loss.backward()
```

This command computes all required gradients for the model parameters using automatic differentiation.

### 4.3 Optimizer and Parameter Updates

Once gradients are computed, model parameters are updated using the Adam optimizer. In basic gradient descent, a parameter  $\theta$  is updated according to:

$$\theta \leftarrow \theta - \eta \frac{\partial \mathcal{L}}{\partial \theta}$$

where  $\eta$  is the learning rate. Adam extends this rule by incorporating momentum and adaptive learning rates, but the underlying principle remains gradient descent.

In this assignment, the learning rate was set to  $\eta = 0.001$ , and training was performed for 5 epochs. The optimizer was defined in PyTorch as:

```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
```

After each backward pass, parameters were updated using:

```
optimizer.step()
```

Before processing the next mini-batch, previously accumulated gradients were cleared to prevent unintended accumulation:

```
optimizer.zero_grad()
```

## 5. Evaluation Methodology

After training, the performance of the neural network was evaluated using a separate test dataset that was not used during parameter optimization. The purpose of evaluation is to assess how well the trained model generalizes to unseen data, rather than how well it fits the training samples.

During evaluation, the model was placed in evaluation mode using `model.eval()`. This ensures that the network operates in inference mode and disables any training-specific behavior. In addition, gradient computation was disabled during evaluation since parameter updates are not required and gradients are unnecessary for forward inference.

For each test sample, the network produces an output vector of logits

$$\mathbf{z} \in \mathbb{R}^{10}$$

where each element represents the score associated with one digit class. The predicted class label is obtained by selecting the index of the maximum logit value:

$$\hat{y} = \arg \max_k z_k$$

This operation selects the class with the highest predicted confidence.

Model performance was quantified using classification accuracy. Accuracy is defined as the ratio of correctly predicted samples to the total number of samples in the test dataset:

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of test samples}}$$

Correct predictions were identified by comparing the predicted labels  $\hat{y}$  with the ground truth labels  $y$ . The total number of correct predictions and total samples were accumulated across all mini-batches in the test set to compute the final accuracy value.

In the implementation, evaluation was performed once at the end of each training epoch. This allowed the evolution of test accuracy to be tracked as training progressed. The evaluation procedure was implemented in PyTorch as:

```
model.eval()
with torch.no_grad():
    outputs = model(images)
    preds = outputs.argmax(dim=1)
```

Disabling gradient computation using `torch.no_grad()` reduces memory usage and computational overhead during evaluation.



The accuracy values obtained at each epoch were stored and later used to visualize model performance over time. This evaluation strategy ensures a clear separation between training and testing, providing an unbiased estimate of the model’s generalization capability.

## 6. Results and Visualization

The training behavior and performance of the neural network were analyzed using two primary metrics: training loss and test accuracy. These metrics were recorded at the end of each training epoch and visualized to assess convergence and generalization behavior. The training loss curve, shown in Figure 3a, exhibits a monotonic decrease across epochs. This trend indicates that the optimization process successfully reduced the cross-entropy loss as training progressed. The sharp decrease in loss during the initial epochs reflects rapid learning of salient features from the input data, while the more gradual decrease in later epochs suggests that the model parameters were approaching a local minimum. The smoothness of the curve indicates stable training without oscillations or divergence, which is consistent with the use of the Adam optimizer and an appropriate learning rate.

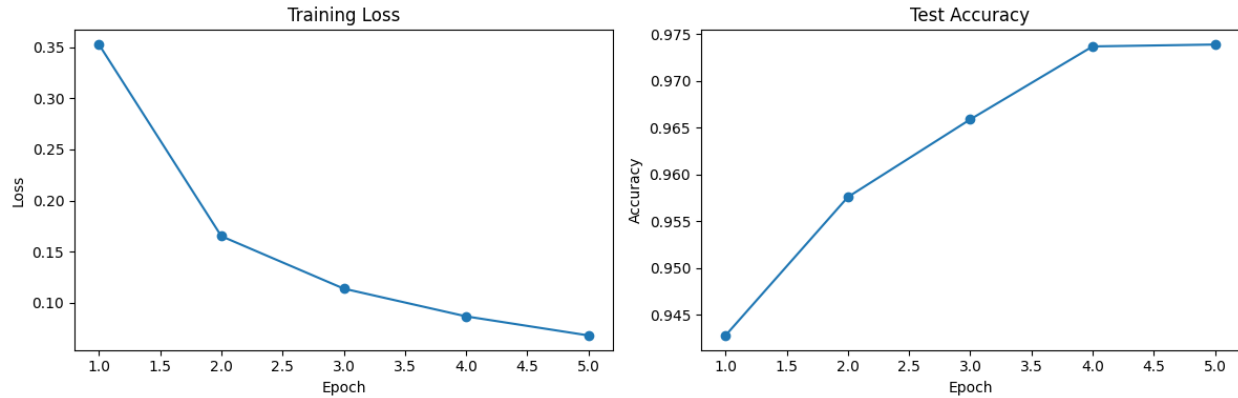


Figure 3: Training loss curve and text accuracy curve

The test accuracy curve, shown in Figure 3b, demonstrates a steady increase over the training epochs, followed by a plateau near the final epochs. This behavior indicates that the model’s ability to correctly classify unseen handwritten digit images improved as training progressed. The stabilization of test accuracy suggests that the network reached its effective capacity given the chosen architecture and training parameters. The absence of a significant drop in test accuracy across epochs indicates that the model did not suffer from severe overfitting. By the final epoch, the network achieved a test accuracy of approximately 97–98%, which is consistent with expected performance for a fully connected neural network applied to the MNIST dataset. This level of accuracy confirms that the model successfully learned meaningful representations of the input data despite its relatively simple architecture. Taken together, the training loss and test accuracy plots demonstrate successful convergence of the learning process and effective generalization to unseen data. The combined analysis of these two metrics provides confidence that the implemented training and evaluation procedures were correct and that the model performance aligns with established benchmarks for this task.

## 7. Discussion and Lessons Learned

This assignment provided practical insight into the end-to-end workflow of training and evaluating a neural network for a supervised classification task. One of the key outcomes was gaining a clearer understanding of how individual components of a deep learning system—such as data preprocessing, model architecture, loss functions, and optimization—interact to produce meaningful results. The fully connected neural network architecture proved sufficient for achieving high classification accuracy on the MNIST dataset. Despite its simplicity, the model was able to learn effective feature representations, as evidenced by the steady decrease in training loss and the high test accuracy. This demonstrates that even relatively shallow networks can perform well on structured and well-understood datasets when trained correctly.

Several common implementation challenges were encountered and resolved during the assignment. These included managing tensor dimensions correctly when flattening input images, ensuring that training and evaluation procedures were properly separated, and correctly tracking performance metrics across epochs. In particular, care was required to evaluate the model once per epoch rather than per mini-batch to avoid incorrect accuracy measurements and plotting errors. Resolving these issues reinforced the importance of maintaining a clear training loop structure and careful bookkeeping of experimental results. Conceptually, the assignment strengthened understanding of how backpropagation and gradient-based optimization operate in practice. Implementing the training loop clarified how forward passes, loss computation, gradient calculation, and parameter updates are coordinated within each iteration. Additionally, working with evaluation mode and disabling gradient computation highlighted the distinction between training and inference, which is essential for reliable performance assessment.

Overall, this assignment reinforced foundational deep learning concepts and provided hands-on experience with PyTorch's model definition and training framework. The lessons learned in this homework form a solid basis for more advanced models and architectures in future coursework.

## 8. Conclusion

This assignment provided hands-on experience with implementing and training a neural network for handwritten digit classification using PyTorch. Through this work, key deep learning concepts such as model architecture design, loss functions, gradient-based optimization, and evaluation were applied and reinforced. The results demonstrate that a simple fully connected network can achieve strong performance on the MNIST dataset, and the concepts learned in this assignment form a foundation for more advanced models and architectures in future coursework.

## 9. AI Usage Disclosure

Artificial intelligence tools were used during the completion of this assignment to support learning, clarification of concepts, and debugging of implementation issues. Specifically, ChatGPT was used as an interactive assistant to help explain deep learning concepts such as neural network architecture, activation functions, loss functions, gradient computation, backpropagation, and evaluation methodology. It was also used to clarify PyTorch-specific implementation details, including tensor shapes, training and evaluation loops, and correct usage of functions such as `loss.backward()`, `optimizer.step()`, `model.eval()`, and `torch.no_grad()`.

In addition, ChatGPT was used to assist in organizing and refining the written report, including structuring sections, improving technical clarity, and ensuring that mathematical formulations and explanations were accurate and consistent with the implemented code. All code used in the final submission was written, reviewed, and executed by the student, and the results presented in this report were generated from the student's own implementation.

The use of AI tools was limited to conceptual guidance, explanation, and editorial assistance, and no AI-generated code or text was submitted without review and understanding by the student.

## References

1. LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.
2. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
3. Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32.
4. GeekySaint, "MNIST Classification with PyTorch," *Kaggle Notebook*. [Online]. Available: <https://www.kaggle.com/code/geekysaint/solving-mnist-using-pytorch>. Accessed: Jan. 2026.